



# Protocol Audit Report

Version 1.0

*CyberGhost*

September 25, 2026

# Protocol Audit Report

CyberGhost

September 25, 2026

Prepared by: CyberGhost

## Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    - \* [H-1] Storing the password on-chain makes it visible to anyone, and no longer private.
    - \* [H-2] `PasswordStore::setPassword` has no access control, meaning a non-owner could change the password.
  - Informational
    - \* [I-1] The `PasswordStore::getPassword` natSpec indicates a parameter that doesn't exist, causing the natSpec to be incorrect.

## Protocol Summary

The PasswordStore protocol is designed as a simple vault intended to allow its deployer (the contract owner) to securely store and update a single private text password on the Ethereum blockchain. Upon deployment, the contract assigns the deployer's address as the owner, who is intended to be the sole party capable of setting new password values and retrieving the stored password via an access-controlled getter function. Intended as a personal password manager on-chain, its core objective is to ensure that confidential credentials remain restricted exclusively to the contract owner while allowing them to update the password whenever needed.

## Disclaimer

CyberGhost makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

## Risk Classification

|            |        | Impact |        |     |
|------------|--------|--------|--------|-----|
|            |        | High   | Medium | Low |
| Likelihood | High   | H      | H/M    | M   |
|            | Medium | H/M    | M      | M/L |
|            | Low    | M      | M/L    | L   |

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

## Audit Details

**The findings described in this document correspond the following commit hash:**

```
1 7d55682ddc4301a7b13ae9413095feffd9924566
```

## Scope

```
1 ./src/  
2 #-- PasswordStore.sol
```

## Roles

- Owner: The user who can set the password and read the password.
- Outsiders: No one else should be able to set or read the password.

## Executive Summary

CyberGhost audited the PasswordStore protocol, uncovering 3 issues (2 High, 1 Info).

The High-severity flaws completely compromise the protocol: any user can read the raw password off-chain via RPC storage reads, and unauthenticated addresses can overwrite the owner's password due to missing access controls. Immediate remediation is required before deployment.

## Issues found

| Severity | Number of issues found |
|----------|------------------------|
| High     | 2                      |
| Medium   | 0                      |
| Low      | 0                      |
| Info     | 1                      |
| Total    | 3                      |

## Findings

### High

#### [H-1] Storing the password on-chain makes it visible to anyone, and no longer private.

**Description:** All data stored on-chain is visible to anyone, and can be read directly from the blockchain. The `PasswordStore:s_password` variable is intended to be a private variable and only accessed through the `PasswordStore:getPassword` function, which is intended to be only called by the owner of the contract.

We show such method of reading any data off-chain below.

**Impact:** Anyone can read the private password, severely breaking the functionality of the protocol.

**Proof of Concept:** (Proof of code)

The below test case shows how anyone can read the password from the blockchain.

1. Create a locally running chain

```
1 make anvil
```

1. Deploy the contract to the chain

```
1 make deploy
```

1. Run the storage tool

We use 1 because that's the storage slot of `s_password` in the contract.

```
1 cast storage <contract address> 1
```

You'll get the output that looks like this: `0x6d7950617373776f726400000000000000000000000000000000000000000000`

You can then parse that hex to a string with:

```
1 cast parse-bytes32-string 0
  x6d7950617373776f726400000000000000000000000000000000000000000014
```

And get an output of:

```
1 myPassword
```

**Recommended Mitigation:** Sensitive data such as passwords should never be stored in plain text on-chain. If the protocol requires off-chain retrieval of the original password, the data must be encrypted

off-chain using asymmetric encryption (e.g., ECIES) or symmetric encryption (e.g., AES) before submitting it to the contract. The smart contract should only store the encrypted ciphertext. Authorized users can then fetch this ciphertext via `PasswordStore::getPassword` and decrypt it off-chain using their private key.

**[H-2] PasswordStore::setPassword has no access control, meaning a non-owner could change the password.**

**Description:** The `PasswordStore::setPassword` function is set to be an `external` function, however, the natspec of the function and overall purpose of the smart contract is that: This function allows only the owner to set a `new` password.

```
1 function setPassword(string memory newPassword) external {
2   @> // @audit EVERYONE is able to change the owner's password.
3     s_password = newPassword;
4     emit SetNetPassword();
5 }
```

**Impact:** Anyone can set/change the contract's password, severely breaking the contract intended functionality.

**Proof of Concept:** Add the following to the `PasswordStore.t.sol` test file:

Code

```
1 function test_anyone_can_set_password(address randomAddress) public
2 {
3   vm.assume(randomAddress != owner);
4   vm.prank(randomAddress);
5   string memory expectedPassword = "myNewPassword";
6   passwordStore.setPassword(expectedPassword);
7
8   vm.prank(owner);
9   string memory actualPassword = passwordStore.getPassword();
10  assertEq(actualPassword, expectedPassword);
11 }
```

**Recommended Mitigation:** Add an access control to the `PasswordStore::setPassword` function:

```
1 if(msg.sender != owner) {
2   revert PasswordStore__NotOwner();
3 }
```

## Informational

**[I-1] The PasswordStore::getPassword natSpec indicates a parameter that doesn't exist, causing the natSpec to be incorrect.**

### Description:

```
1      /*
2      * @param newPassword The new password to set.
3      * @audit There is no newPassword parameter.
4      */
5      function getPassword() external view returns (string memory)
```

The `PasswordStore::getPassword` function signature is `getPassword()` while the natSpec says it should be `getPassword(string)`.

**Impact:** The natSpec is incorrect.

**Recommended Mitigation:** Remove the incorrect natSpec line.

```
1 -      * @param newPassword The new password to set.
```